



**DE MONTFORT
UNIVERSITY
LEICESTER**

CTEC3451 Development Project

Lone Wilderness

First Deliverable

Project Supervisor: Conor Fahy

Project Author: Daniel Bullin (P14132258)

Literature review

Terrain rendering is, perhaps, one of the most important components of a video game. The rendering of terrain defines the game world and the combination of the terrain and the objects in the game world expand that definition and can make or break an experience for a player. Video games are not the only application of terrain rendering, “Virtual terrains have diverse applications in many fields, ranging from movies and video games to simulation.” (Vitacion and Liu, 2019). With such a large array of applications coupled with the complexity of such a topic, “the creation of three-dimensional virtual terrains is a constantly evolving topic in the field of computer graphics.” (Vitacion and Liu, 2019).

Environments handmade by artists has been the norm in video games for many years, and throughout the history of video games we have been introduced to many beautiful environments which are the product of such an approach. A few notable games to show off the power of handmade environments are GTA 5, Red Dead Redemption, Skyrim and the soon to be released, Cyberpunk 2077. There are, however, a few disadvantages for this approach from both the development side and gameplay side. Regarding the development side, “The creation of realistic and visually pleasing virtual terrains involves the addition of several types of features, which increase the overall complexity of the task. Roads, railways, and rivers are usual features that demand significant manual work” (Nascimento, Pozzer and Franzin, 2019).

“Procedural Terrain Generation (PTG) can streamline much of this work through the use of pseudo-random algorithms. The term PTG refers to the creation of content by an automated system, rather than being produced manually.” (Rose and Bakaoukas, 2016). There are many different techniques to choose from which can produce such algorithms, each with their pros and cons and to this day, this area of game development is constantly evolving. One approach, and the chosen technique that will be implemented in my development project, is noise, other approaches include “software agents, evolutionary algorithms and erosion modelling techniques.” (Rose and Bakaoukas, 2016).

“The terrain data produced by a noise function can both be rendered directly and/or used to produce a heightmap. A heightmap is an image that can be used to store the elevation value of each point of the terrain using one or more colour channels.” (Rose and Bakaoukas, 2016). “When generating heightmaps, the input to these noise functions are the coordinates of each vertex and the output is each vertex’s elevation value.” (Vitacion and Liu, 2019). Greyscale heightmaps interpret the luminosity of the texture as the elevation data, white being the highest point and black being the lowest. As mentioned previously, different noise functions exist, each with their advantages and disadvantages, the figure below illustrates these, courtesy of Thomas J. Rose and Anastasios G. Bakaoukas.

TABLE I. NOISE FUNCTIONS

Algorithm	Speed	Quality	Memory Requirements
Diamond-Square Algorithm	Very Fast	Moderate	High
Value Noise	Slow - Fast*	Low - Moderate*	Very Low
Perlin Noise	Moderate	High	Low
Simplex Noise	Moderate**	Very High	Low
Worley Noise	Variable	Unique	Variable

*Depends on what interpolation function is used

**Scales better into the higher dimensions than Perlin Noise

Figure 1. (Rose and Bakaoukas, 2016)

As Figure 1 suggests, each algorithm has a description about three categories: speed, quality and the memory requirements. The algorithm of choice depends on the application usage, and with the rising demand of high-quality graphics in video games, I will be implementing Perlin noise in my project for its high quality coupled with its low memory requirement. I sacrifice some speed advantages, but it is a worthwhile cost to gain high grades in the other two categories.

A brief overview of some of the other techniques such as: “Software agents provide an interesting alternative to noise functions for generating terrain. One of the great advantages they yield is that they are much controllable and can be used in a PTG system that works with designer-chosen parameters, rather than abstract random number seeds” (Rose and Bakaoukas, 2016). Additionally, using the evolutionary algorithms “are somewhat more difficult to control but provide a valuable way of generating a broad spectrum of game content types. The initial content population and fitness function are the best ways an evolutionary algorithm can be regulated along with reducing the amount of mutation that takes place each generation.” (Rose and Bakaoukas, 2016). Furthermore, an advantage for the evolutionary algorithms, which I will now refer to by EA, is that it gives you more control over the result. “By using EA, more control can be exerted over the generation process, allowing developers to get more desirable results or for maps to be automatically generated to meet player preferences” (Raffe, Zambetta and Li, 2012). A downside is that EA is still relatively new, having only emerged in the field in recent years. A downside of the software agents is the real-time requirement if the game needs to offer on-the-fly generation of the map such as infinite terrain. (Doran and Parberry, 2010).

As mentioned previously, noise functions can produce a texture known as a height map. “A Height Map also known as a Digital Terrain Model (DTM) is a collection of approximate elevation levels for a set of discrete points on a grid. These heights are vertical distances between terrain points and a reference surface like mean sea level” (Emmanuel et al., 2019). Figure 2 below shows an example of a height map that is often greyscale. Textures like these are used to describe the elevation of terrain or even displacement in geometry for techniques like displacement mapping. Figure 3 shows the result of using a plain flat mesh in conjunction with a height map to produce more realistic terrain.

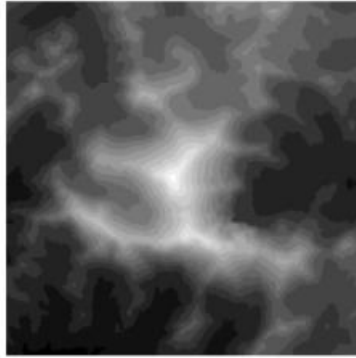


Figure 2. (Emmanuel et al., 2019)

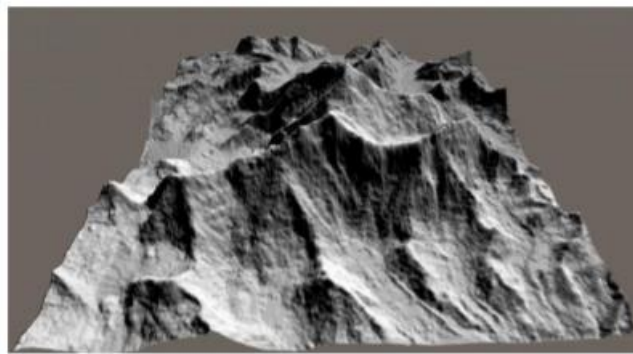


Figure 3. (Emmanuel et al., 2019)

Back to Perlin noise, “2D Perlin noise map, created from fractal Brownian Motion (fBM) is a series of undulating data that has smoothness and predictability. This creates a far superior representation of a landscape with mountains and valleys.” (Emmanuel et al., 2019).

Shadows

“Shadow plays an important and unique role in applications of real computer graphics. Besides dramatically improving facticity of rendered images, shadow also provides users some visual cues about information of scene, such as shapes of geometries, relative positions between objects, complexity of shadow occluders, and so on.” (Liu and Pang, 2009).

“Real-time rendering of shadows is a fundamental issue in computer graphics and virtual reality technology.” (Xiao, Han-jun, Shi-dong, 2010). There are two commonly used techniques to add shadows to a rendered scene, shadow maps and shadow volume. Shadow mapping was a concept first introduced to us by Lance Williams in his 1978 paper entitled “Casting curved shadows on curved surfaces”. The concept of shadow mapping is simple, the idea is based on rendering the scene from the viewpoint of a light source and storing the distances between the light source and object to a texture commonly referred to as a shadow map. “we can calculate the distance to the light source and compare it with the distance value recovered from the shadow map.” (Wiens and Domik, 2015). Figure 4 illustrates two different points of reference, one being in shadow and the other not. The

determining factor whether a point is in shadow is if the distance from the viewpoint to the object point is greater than the distance stored in the shadow map.

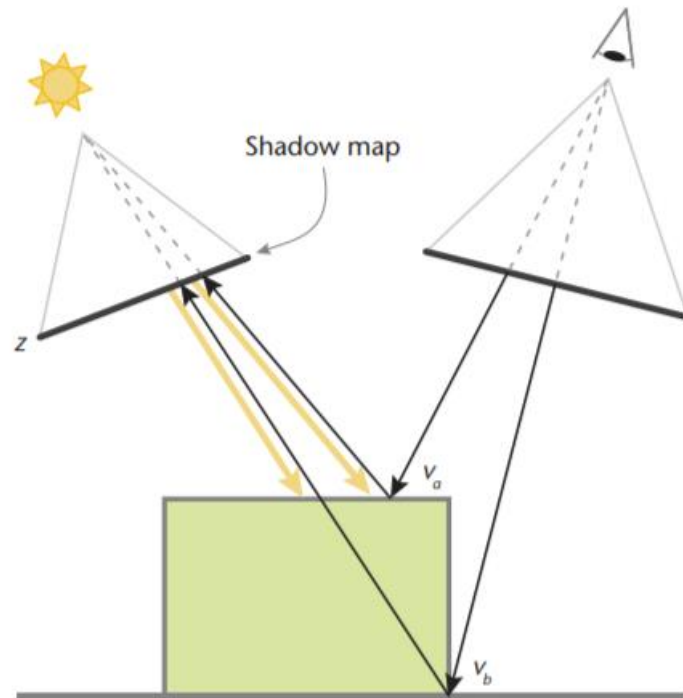


Figure 4. (Wiens and Domik, 2015)

There are, of course, a few artefacts we must deal with in order to improve the quality of shadow we get through shadow mapping. “Regular shadow mapping suffers from the so-called “shadow acne” artifact, which refers to erroneous self-shadowing due to precision issues”. (Annen, et al, 2008). Shadow acne is usually the result of the limitation of resolution of the shadow map. Multiple fragments can sample the same value from the depth map when they are relatively far away. A solution to shadow acne is to apply a bias, we slightly offset the depth of the surface by a small amount. An undesired consequence of applying this bias is peter panning. A slight visible offset of shadows to the actual objects becomes evident, however this can be fixed with front face culling when we generate the shadow map.

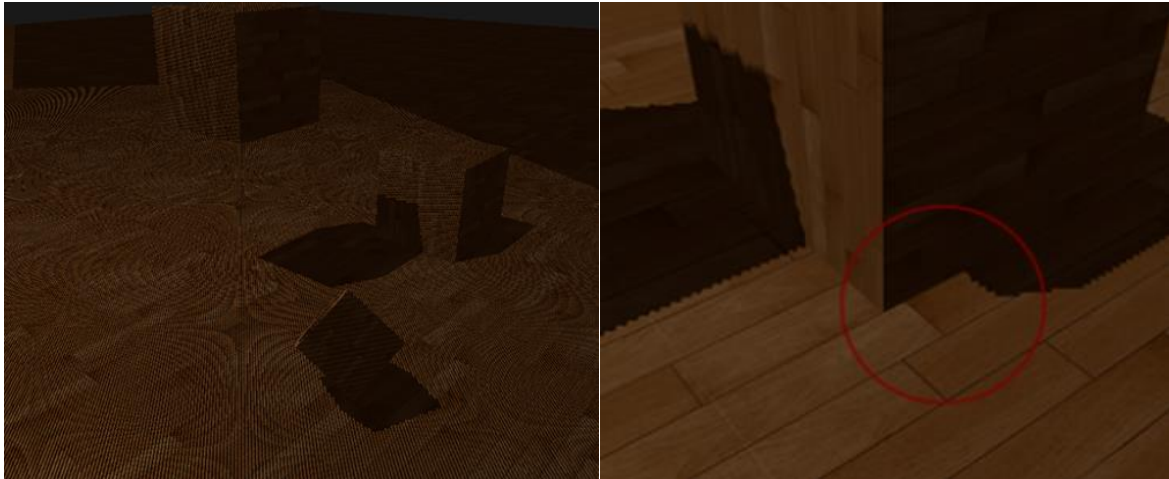


Figure 5. Displaying shadow acne and peter panning (de Vries, 2016)

The other commonly used algorithm for real time shadow rendering is shadow volumes. “Shadow volumes algorithm is one of the commonly used real-time soft-shadow rendering methods. Shadow volume is a 3D space forged at the back of the objects because of these opaque objects’ shading of the light. It is determined by the position relationship between the light and the occluder. The original shadow volumes algorithm is presented by Crow (1977) and first implemented by Heidmann in 1991.” (Liu and Pang, 2009). The idea of shadow volumes is that you project a ray from the light source through each vertex of the current geometry being drawn. These projections will form a volume and any point inside this volume can be considered ‘being in shadow’. “To determine whether an object **C** in the scene is visible, we follow a ray through a pixel until the ray hits the object, and calculate the number of faces of the shadow volume it crosses. While the ray is on its way to this object, we increment a counter in the stencil buffer each time it goes into a shadow volume and decrement the counter whenever the ray goes out of a shadow volume until it reaches **C** after crossing every volumes. Finally, if the counter is greater than zero, then it indicates that the number of going into the volumes is larger than the number of going out of them, that is, **C** is located in shadow; otherwise, **C** is located outside the shadow” (Liu and Pang, 2009).

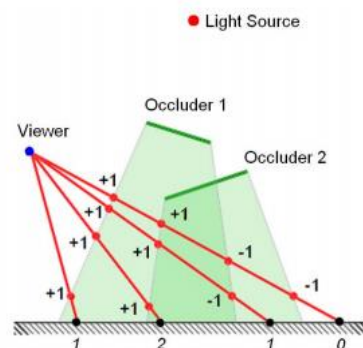


Figure 6. Shadow Volumes (Liu and Pang, 2009)

Water

Water is a detail commonly found in games, usually a part of the environment. Water is also one of the environmental details developers spent a lot of time working on to produce realistic water graphics and animation.

The geometry of a water model is generally a plane, but the details are considered for the height value. “In 2002, Isidoro et al. proposed to use the sum of four sine functions in a vertex shader to describe the height and orientation of water surface. In the same year, Laeuchli presented a shader calculating surface height using three Gerstner waves, but it would consume large amounts of system resources when we simulated large bodies of water (such as Ocean etc.) on the CPU. But nowadays we can make full use of the advantages of parallel computing of GPU to reduce the demand of system resources.” (Li and Xu, 2009). Being that water has such simple geometry, the only real calculation is, as mentioned previously, is the height value. One approach can be modelling the surface of water body can be modelled using the sine wave. A sine wave is a geometric wave that oscillates over time. Figure 7 is an equation for calculating the height of each vertex in the geometry of the water model.

$$W(x, y, t) = A \times \sin(D \cdot (x, y) \times \omega + t \times \phi)$$

Figure 7. Equation for calculating the height value of a vertex using sine waves (Li and Xu, 2009)

In Figure 7, “W is the height value for each vertex. It is the function of the vertex position in the form of x and y coordinates values and the time t. A (Amplitude) is the height from the water plane to the wave crest. D (Direction) is the horizontal vector perpendicular to the wave front along which the crest travels. is the speed of wave propagation.” (Li and Xu, 2009).

“If the water wave has a wavelength of L, we can easily calculate the wave speed at which it progresses across the surface. The dispersion relation for water (Tessendorf 2001), ignoring higher-order terms, is shown as the following equation:”

$$\omega = \sqrt{g \times \frac{2\pi}{L}}$$

Figure 8. Calculating the wave speed. (Li and Xu, 2009)

“Where, is the wave speed and g is the gravitational constant.” (Li and Xu, 2009). This is a good method for calm body of water, other algorithms exist for more rough bodies of water. There are other ways of rendering bodies of water including using displacement maps.

References

- Vitacion, R.J., and Liu, L., (2019) "Procedural Generation of 3D Planetary-Scale Terrains," *2019 IEEE International Conference on Space Mission Challenges for Information Technology (SMC-IT)*, Pasadena, CA, USA, 2019, pp. 70-77.
- Torres do Nascimento, B., Tadeu Pozzer, C., and Paulus Franzin, F., (2019) "Procedural Editing of Virtual Terrains Using 3D Bézier Curves," *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, Rio de Janeiro, Brazil, 2019, pp. 135-143.
- Rose, T.J. and Bakaoukas, A.G., (2016) "Algorithms and Approaches for Procedural Terrain Generation - A Brief Review of Current Techniques," *2016 8th International Conference on Games and Virtual Worlds for Serious Applications (VS-GAMES)*, Barcelona, 2016, pp. 1-2.
- Raffe, W.L., Zambetta, F. and Li, X (2012), "A survey of procedural terrain generation techniques using evolutionary algorithms", *2012 Congress of Evolutionary Computation (CEC 2012)*, United States, 10-15 June 2012, pp. 2090-2097.
- Doran, J. and Parberry, I., (2010), "Controlled Procedural Terrain Generation Using Software Agents", *2010 IEEE Transactions on Computational Intelligence and AI in Games*, vol. 2, no. 2, pp. 111-119.
- Emmanuel, K.S., Mathuram, C., Priyadarshi, A.R., George, R.A. and Anitha, J., (2019) "A Beginners Guide to Procedural Terrain Modelling Techniques," *2019 2nd International Conference on Signal Processing and Communication (ICSPC)*, Coimbatore, India, 2019, pp. 212-217.
- Liu, N., and Pang, M., (2009) "A Survey of Shadow Rendering Algorithms: Projection Shadows and Shadow Volumes," *2009 Second International Workshop on Computer Science and Engineering*, Qingdao, 2009, pp. 488-492.
- Li, X., Jin, H., Mei, S., (2010) "GPU-accelerated method for real-time shadow generation," *2010 2nd International Conference on Computer Engineering and Technology*, Chengdu, 2010, pp. V6-656-V6-659.
- Wiens, A. and Domik, G., (2015) "In-Class Exercise for Shadow Mapping Algorithms," in *IEEE Computer Graphics and Applications*, vol. 35, no.3, May-June 2015, pp. 15-19, Available from: <https://www.computer.org/csdl/magazine/cg/2015/03/mcg2015030015/13rRUx0xPLb> [Accessed 26/11/2020].
- De Vries, J., (2016) Shadow Mapping. [Online]. Available from: <https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping> [Accessed 26/11/2020].
- Annen, T., Mertens, T., Seidel, H., Flerackers, E. and Kautz, J., (2008) "Exponential Shadow Maps" *2008 Proceedings of graphics interface*, Ontario, Canada, 2008, pp. 1-7.
- Li, X. and Xu, M., (2009) "Water simulation based on HLSL," *2009 IEEE International Conference on Network Infrastructure and Digital Content*, Beijing, 2009, pp. 1066-1069.

Functional Requirements

Functional requirements are the product features that enable the end user to accomplish their goals. In other words, they describe the functionalities of a piece of software. As described by Garima (2019), they will define the functionality of the project implementation but also what the system must not do. “Additionally, while functional requirements define what the system does or must not do, non-functional requirements specify how the system should do it.” (QRA Corp, 2019). Pertaining to my project, a video game only has one type of system user, the player, whose goal is defined by the objectives of the game. I will be discussing both the functional and non-functional requirements of the project to highlight the design of the project implementation.

Lone Wilderness is a 3D sandbox game. A sandbox game is a game that gives the player freedom to decide how to complete objectives, some sandbox games may have no objectives. Lone Wilderness contains procedurally generated terrain which the player can traverse. The terrain contains randomly generated resources such as trees and rocks which the player can interact with to collect items which can be used for crafting. Crafting requires a set of materials to make items, the player uses their materials they collect from resources found in the world. The player can use these items they create to aid in further resource collecting or kill entities that exist in the world. The player can also place structures in the world such as walls or platforms. There is no end goal, the player simply must survive in the world by collecting resources and killing entities.

Another consideration is that I will be creating my own game engine to create Lone Wilderness so the functional requirements identify the requirements of not just the game but also what the game engine must support.

Functional Requirements

The following list will outline the functional requirements regarding navigation and data outside the game session:

- The application will allow a user to continue game sessions
- The application will allow a user to create a new game session
- The application will allow a user to see and change the settings
- The application will allow a user to see the credits
- The application will allow a user to stop the program
- The application will allow a user to pause a game session
- The application will provide a user the ability to exit a game session
- The application will provide a user the ability to save a game session
- The application will provide a user the ability to change the key binds to their own configuration
- The application will provide a user the ability to change the mouse sensitivity of the player camera rotation
- The application will provide a user the ability to change the field of view

The above requirements dictate the behaviour of the application outside a game session which allows the user to create or continue an existing game session, which is common practise on most games. The ability to allow the user to change settings such as key binds or values such as mouse sensitivity is present in all present-day games and the project must allow the user this luxury. The ability to let the player exit the application is self-explanatory. The player should be able to pause and exit a game session, saving the session data if necessary.

The following list will outline the functional requirements in a game session:

- The game will allow the user to control a player game object
- The game will allow the user to switch camera perspectives focused on the player game object
- The game will allow the user to traverse the randomly generated terrain when using the movement interface controls
- The game will allow the user to jump
- The game will provide UI to show player data such as health, attack damage, etc
- The game will allow the presence of resource game objects such as trees, rocks, grass, etc
- The game will allow the user to interact with other game objects in the scene when in proximity
- The game will allow the presence of non-interactable game objects in the scene
- The game will allow the user to open a crafting UI
- The game will allow the user to craft items with items they have acquired
- The game will allow the user to open an inventory UI
- The game will allow the user to have a hotbar for quick access to items
- The game will allow the user to place items into their hotbar
- The game will allow the presence of non-player entities
- The game will allow the user to fight the non-player entities
- The game will allow the non-player entities to drop items upon death
- The game will allow the user to place structures down into the world (Walls, platform, torch, etc)
- The game will allow the presence of basic physics
- The game will allow the simulation of gravity in the world
- The game will allow the presence of a skybox
- The game will allow the presence of clouds
- The game will allow the presence of light objects which will represent an accurate model of light
- The game will allow the presence of various weather
- The game will allow the presence of shadows cast by game objects in the world
- The game will allow the presence of water in the world

A use case diagram is a type of UML (Unified Modelling Language) diagram belonging to the behavioural set of diagrams. Its primary use is specifying the expected behaviour of a piece of software. Its focus is on what it can do and not how it does it. I will be using Lucidchart to create a set of use case diagrams which visualises the expected functionality of the game application, in accordance with the list of functional requirements above. The primary actor is the player, they are the only actor in the various systems for this project.

Main Menu – The scene the player navigates to continue game sessions, create new ones, change game settings, view the credits, or exit the game application.

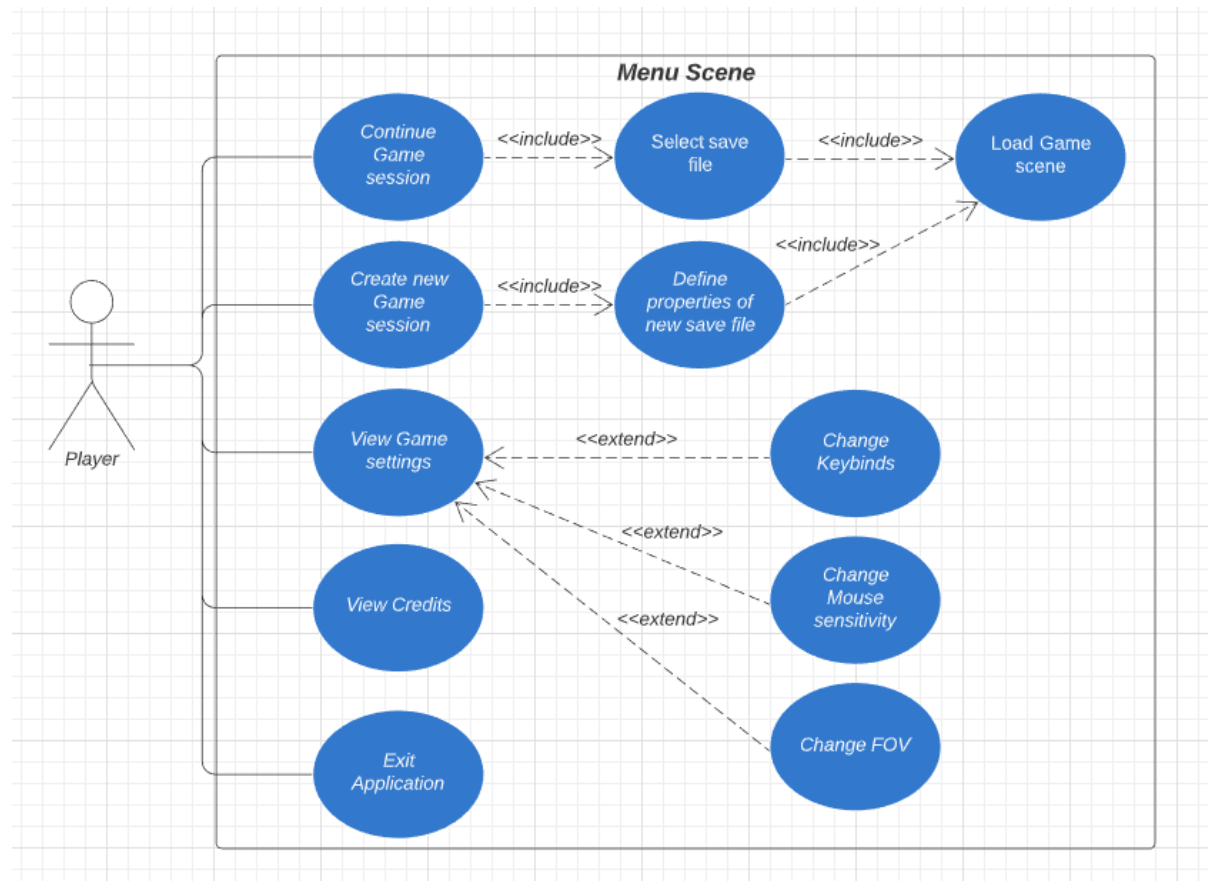


Figure 9. Use case diagram for the Main Menu scene

Game Scene – The game scene is the main scene in the project. The player can traverse the environment that is being generated. They can control a player character and interact with various other game objects. The diagram represents most of the list mentioned above and shows the sophistication the implementation will be to satisfy the functional requirements.

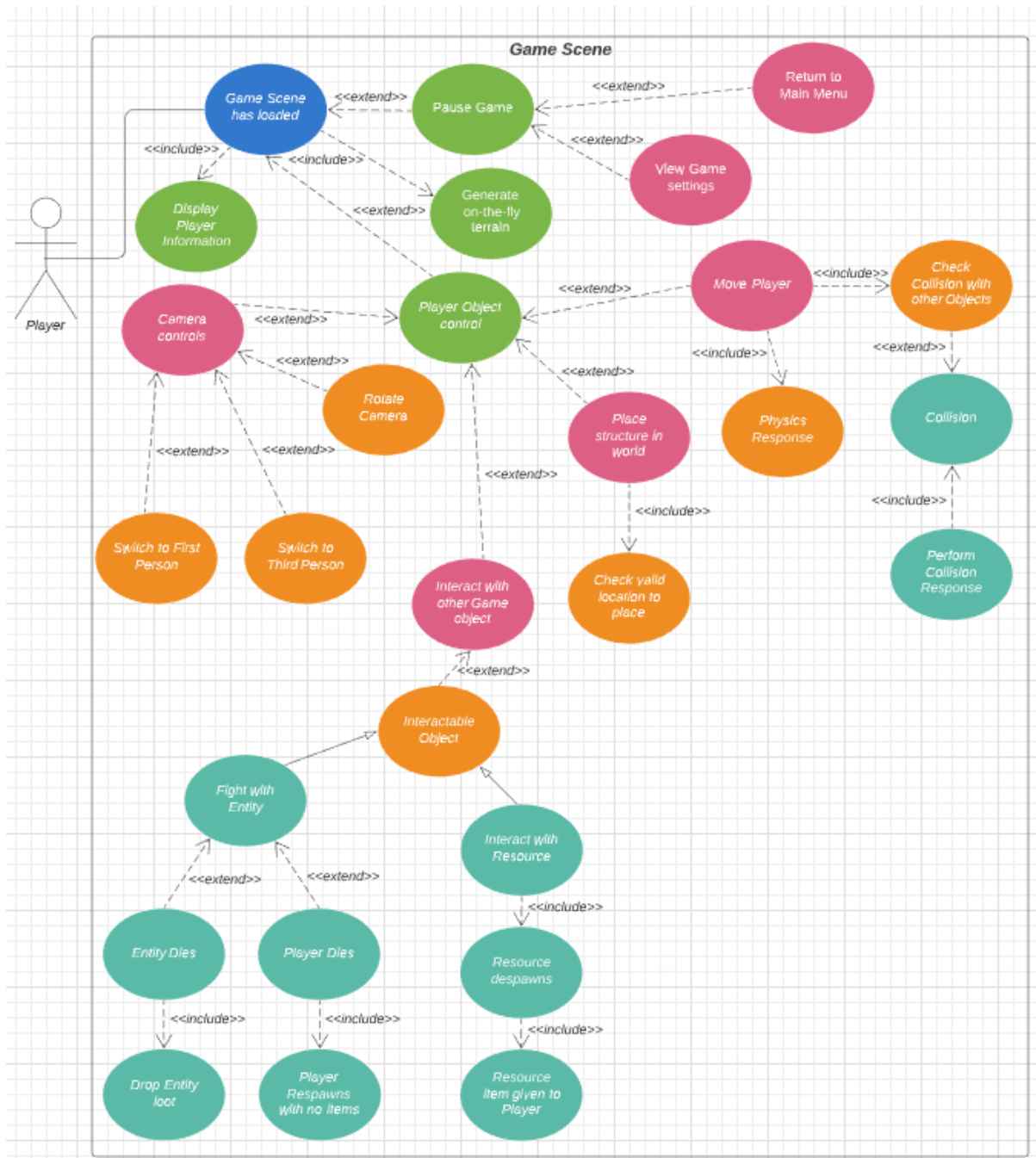


Figure 10. Use case diagram for Game scene

Non-functional Requirements

Non-functional requirements describe how the system will work in relation to the functional requirements discussed previously. The most common types of requirements to consider: Performance, scalability, portability, compatibility, reliability, availability, maintainability, security, localization, and usability.

The following list contains some of the non-functional requirements demanded by the project:

Performance

- The game will run at least 60 FPS on most systems
- The game will load and unload system resources depending on the current area of play (i.e Unloading far away game terrain, Entities too far away, etc) for better performance and memory consumption
- The game will be multi-threaded which allows better performance and faster completion of system tasks
- The game will allow better performance on better systems by allowing performance related settings to be changed

Usability

- The game will offer personalized game settings which the user can change
- The controls are intuitive and common amongst most games
- The UI is simple and clear to the user

Portability and Compatibility

- The software uses OpenGL version 4.5 which is cross-platform
- The software uses GLFW which is a cross-platform library
- The software uses assimp which is a cross-platform library
- The software uses a JSON third party library which is cross-platform
- The software uses GLAD an OpenGL loading library which is multi-language
- The software uses GLM a mathematic library which is cross-platform
- The software uses a third-party logging system which is cross-platform
- The software requires the user to have a graphics card which implements OpenGL
- The software requires the user to have a mouse and keyboard

Localization

- The game text is in English

Security

- No important personal user data is stored by the system
- All game data is stored locally and has no value outside the game

Reliability, Availability and Maintainability

- The game will handle all potential crashes without causing the system to crash and report the issues to a log file
- The software prints to console various system information during debug mode for easier debugging
- The game will not require an internet connection to play for larger availability
- The software is built with compartmentalised systems for easier maintainability

References

GARIMA. (2019), Why Business and Functional Requirements are Vital for a Project's Success, Product Development, [Online]. Available from: <https://www.netsolutions.com/insights/business-and-functional-requirements-what-is-the-difference-and-why-should-you-care/> [Accessed 01/01/21].

QRACORP, (n.d.) Functional vs Non-Functional Requirements: The Definitive Guide, [Online] QRACorp. Available from: <https://qracorp.com/functional-vs-non-functional-requirements/> [Accessed 01/01/21].

System Design & Documentation

System Design Document (SDD)

The purpose of a system design document, here forth called SDD, is to proficiently describe how the functional and non-functional requirements, spoken about previously, are transformed into a more high-level technical system design specification. The document contains the design goals and considerations, provides an overview of the system architecture, describes the data format associated with the system, the format of user input and user interface. The following paragraphs present the features for this project and provide a clear understanding of the system design and goals to take forward into the implementation stage.

System Overview

Lone Wilderness is a 3D sandbox game that aims to provide players with the ability to traverse and survive in a world with procedurally generated and realistic terrain. Players can interact with the environment and collect resources from the many resources nodes that are generated in the world, and these resources can be used as materials which can be used in the crafting system to produce other more valuable items which have an impactful use in the game. These include from allowing the player to build structures in the world to creating tools or weapons to increase the proficiency in tasks the player can complete. The player can attack and kill the various non-player character entities that spawn which provide unique items that are useful to the player. The game offers a lot of customization from different camera perspectives, to custom player controls – changeable for the player's comfort. The game requires a lot of incredibly sophisticated software requirements and I will not be using a commercial game engine to create the game, I will, in fact, be using my own game engine. This decision therefore requires the initial stage of implementation for this project to work on the engine to support the game features I desire for Lone Wilderness. The design goals and system architecture will discuss the design of the engine further.

Design Assumptions

The system will be designed on a windows OS. I mentioned the various library dependencies of the project in the non-functional requirements section and while all libraries but one was explicitly labelled as cross-platform, I will not be testing the different OS so the assumption that issues may rise or is failure to run on other OS is, at this point, to be assumed. However, throughout the development of the project, new dependencies may be required that may not be cross-platform. Console or mobile platforms will not be supported and as it is a PC platform, a mouse and keyboard are a requirement to interact with the controls of the game and therefore the design assumes that players will not have any physical disabilities that hinder the use of a mouse and keyboard. Another hardware assumption is the graphics API I will be using for development is OpenGL and therefore the graphics card in the player's PC must support this API. The game will be single player so an internet connection will not be required to play the game. As regular testing of the implementation occurs, various aspects will be highlighted that require change or a completely different implementation so

we must assume the functionality of the project will change because of a software side problem or a testing of the feature in-game does not give the desirable effect.

Design Constraints

On the hardware side of testing, I only have access to my own personal computer and laptop. This presents the issue of testing the implementation of the project on different hardware capabilities regarding performance. The current state of the world given current real-life events such as COVID has made gaining access to further hardware for testing more difficult such as university hardware. So, guarantees regarding performance or an accurate minimum hardware specification cannot be assured. Additionally, with the nature of procedurally generated terrain, testing an infinite number of outcomes is infeasible so every eventuality cannot be accounted for. A large sample size would result in a confident conclusion of the success of a feature such as this however there is scope for missing something. Procuring assets such as textures and models is a time-consuming task especially for someone that is not artistically gifted, and given the time constraint of the project, the graphics is an area I place low on the priority list given my area of expertise and desired career path.

Design Goals

There are a lot of goals to produce the system designed to fulfil the project contract and system design. Lone Wilderness is a 3D sandbox game produced using my own game engine which is written in C++ and uses OpenGL as the graphics API. Goals must be met for both the sandbox application and the engine library.

Engine Goals

- Implementation of system manager which manages all systems
- Timing system which provides an interface to measure time
- Window system which provides an interface to create and manage a set of windows
- Event Manager system which provides a centralized location to handle and manage various user input events
- Resource Manager system which provides a centralized location to manage and store various resources used by the sandbox application
- Rendering system which handles both 2D and 3D rendering in a batched system (Increased performance)
- Material system which handles how we describe how to texture models and the physical properties of models
- Scene Manager system which can create and manage a set of scenes
- Entity definition and their components which populate scenes (Game objects such as Player, trees, rocks, terrain, etc)

Sandbox Game Goals

- Player object control

- Camera control
- Game object interaction
- Scene navigation and transitioning
- UI creation and interaction
- Game settings configuration
- Procedurally generated terrain
- Entity spawning and AI
- Crafting system
- Inventory system
- Ability for player to place structures into the world
- Day/Night graphics transition
- Ability to pause the game
- Good strong performance and memory consumption

System Architecture

The system architecture will be straight forward, with the project being such a huge undertaking, keeping a straight away architecture is imperative to producing a robust system. I will be employing an object-oriented approach with the classes with each class representing a model which serves some purpose. I will be creating various systems which will have its own abstract class type with other system managers inheriting this to enforce a structure which will be the same across all systems such as: Scene Manager, Resource Manager, Window Manager, Event Manager, etc. Through inheritance, each system will have the same functionality regarding its system usage stored in a single list in the System Manager class through polymorphism. These system classes will be implemented as static classes as only one manager of a system type should be created, there should never be two Scene Managers, for instance.

I plan to make use of the ECS (Entity component system) architectural design regarding the game objects in the scene as it is a familiar and popular methodology for defining the contents of the scene.

I also will be producing many setter and getter methods in classes to enforce encapsulation of data to ensure the robustness and security of data in the project can only be manipulated by approved parties.

The engine itself will be produce in a different project than the sandbox game but a part of the same solution. This allows me to produce a game engine that is a standalone product which my sandbox will make use of. This is ideal as the engine can then be used in other projects without it being directly tied in with the sandbox application.



Figure 11. UML Class Diagram

Basic Game Design Document (GDD) & UI Design

The following paragraphs attempt to discuss the contents of the Lone Wilderness including the design of all UI with associated design choices. Given, the time restraint - I cannot add all that I wish to add but the figures below detail large details that satisfy the game functional requirements and provide detail for implementation.

The first UI the player will encounter is the loading screen. This is a simple communication attempt to the player that the system is loading the resources it will need to function, this process may take a while given the scale of resources being loaded.

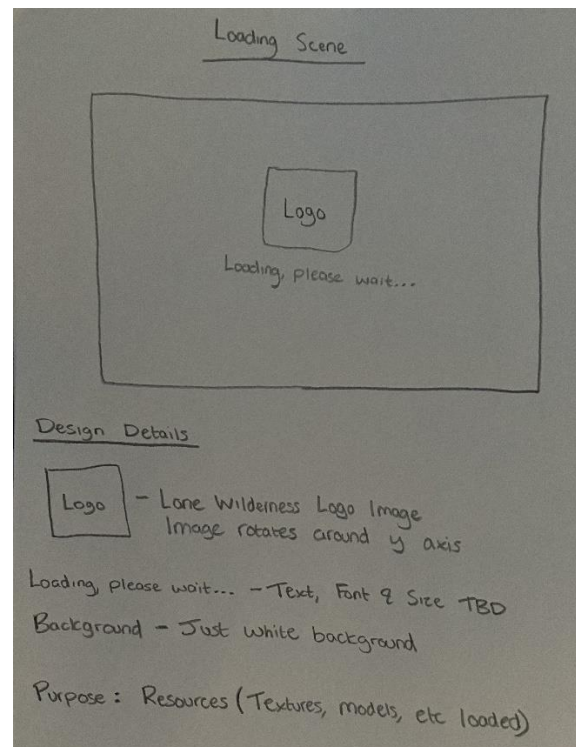


Figure 12. The Loading screen UI

The loading UI is simplistic in design, showing the logo for the game which will rotate around the y axis as the resources are being loaded. The background is just a clear colour of white. The UI's purpose is to just illustrate to the player that the system is currently loading the resources it will need later.

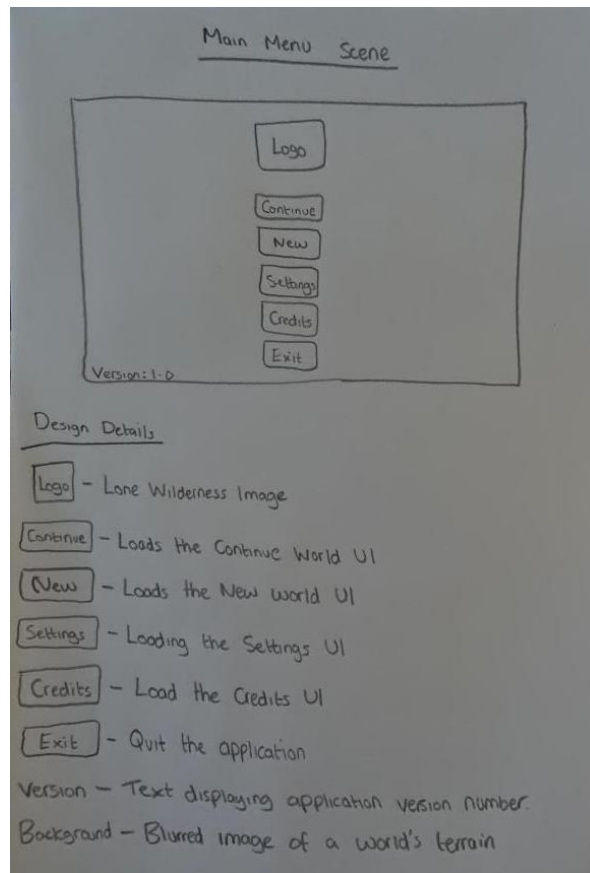


Figure 13. The main menu UI

The main menu provides a variety of options to the player. They can continue a save file they currently have, create a brand-new game save, view and change game settings, view the credits of the project or exit the application. Each of these options are available to the player through a button which can be pressed. The logo is present at the top of the UI, this seems industry standard as most games provide some identity of the game on their main menu. A small text object in the bottom left provides the version number of the game which provides a nice little helper to identify the version they are currently playing. The background will be a still image of an example terrain in the game world with the blurred effect.

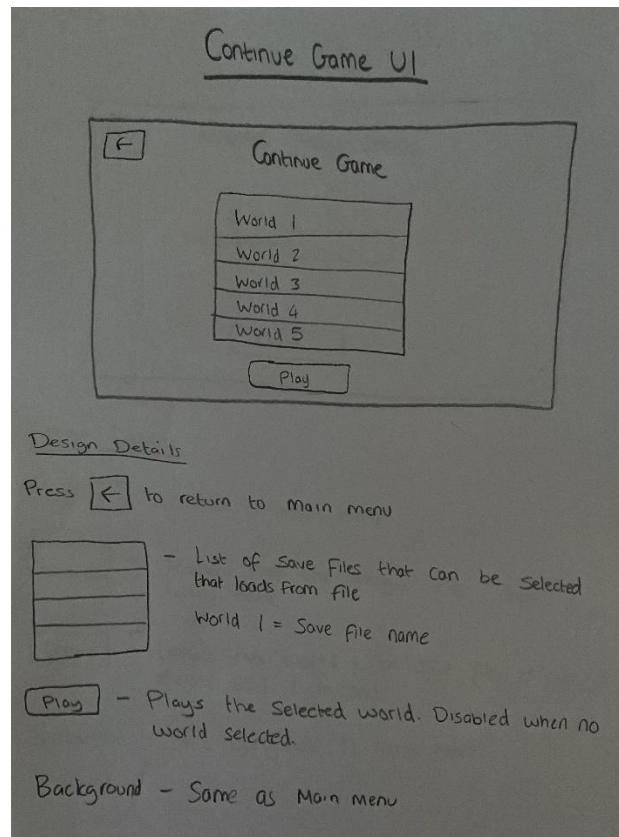


Figure 14. The continue game UI

The continue game UI consists of a list of save files the player has created. They can select one of these save files and hit the play button to load the save file. A back button has been added in the top-left to return to the main menu.

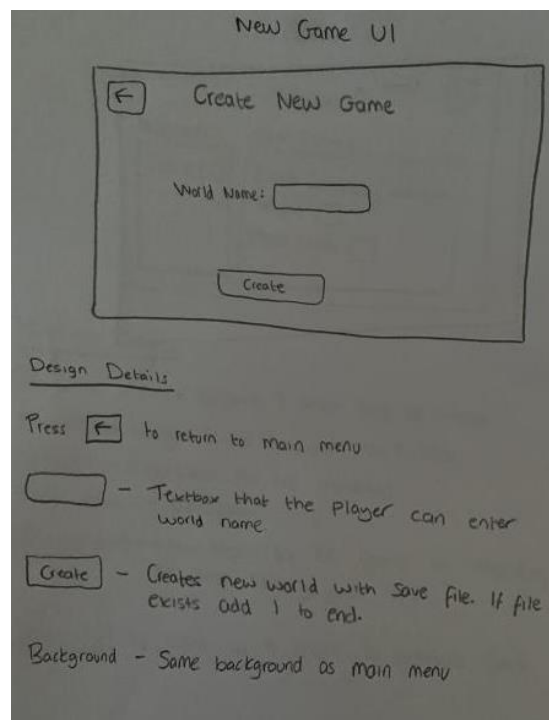


Figure 15. The new game UI

The new game UI uses a simple design allowing the player to name their world through a textbox. The player can then create the world, checking if the save file name has been used, a number at the end of the save name will be added if a file exists. Like the continue game UI, a previous button will return the player back to the main menu.

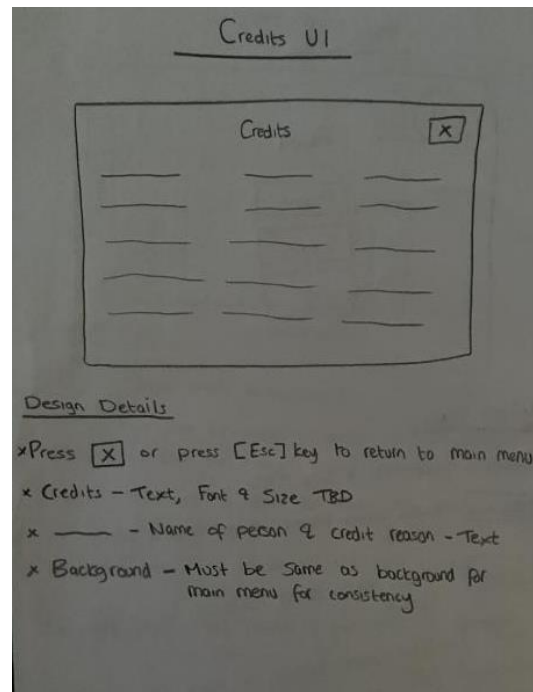


Figure 16. The credits UI

The credits UI will be a wall of text labelling the author name and the reason for the credit. The close button will allow the player to return to the main menu.

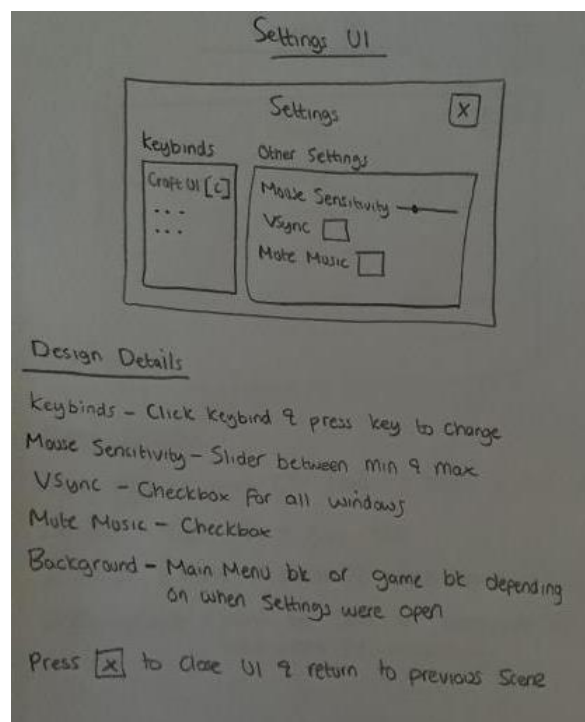


Figure 17. The settings UI

The settings UI contains a list of all the key binds available in the game. The player can click this text and press a key to overwrite the current key bind to the key that was just pressed. A slider will be available to change the mouse sensitivity. Checkboxes will allow the music to be muted or enable VSync across all windows.

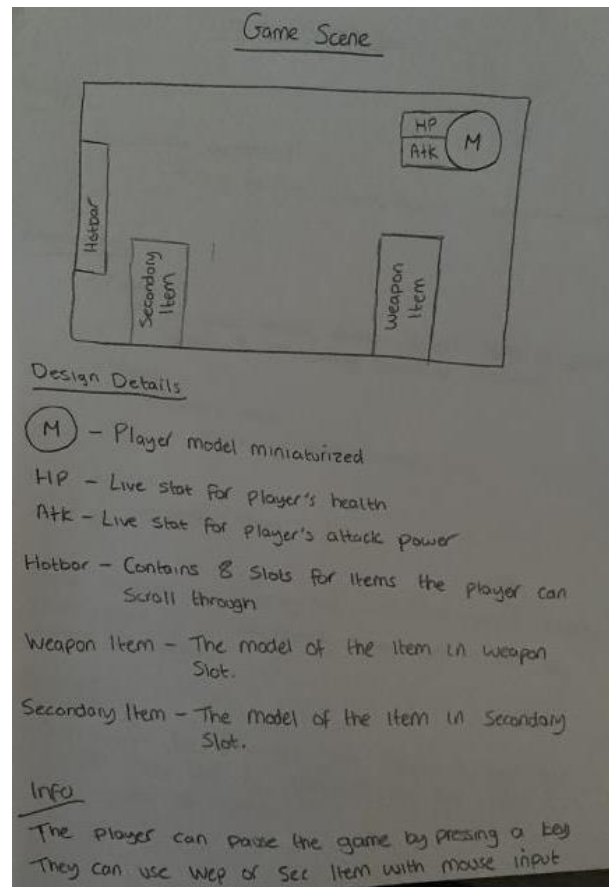


Figure 18. The game scene UI

The game scene UI is the most important and most used. This will be the UI present when the game is currently playing. The top right of the screen will see a miniaturised model of the player with their health and attack power stat next to the model representing “the player”. A hot bar will be on the left which makes up of 8 items which the player can use. Scrolling with the mouse will scroll through the list. The weapon item and secondary item will be the models of the items currently in the associated slots which can be done through the inventory UI. There will be some key binds on this scene to allow access to the crafting menu and the inventory system. Overall, the UI was designed to be clutter-free to maximize the amount of world space seen by the player.

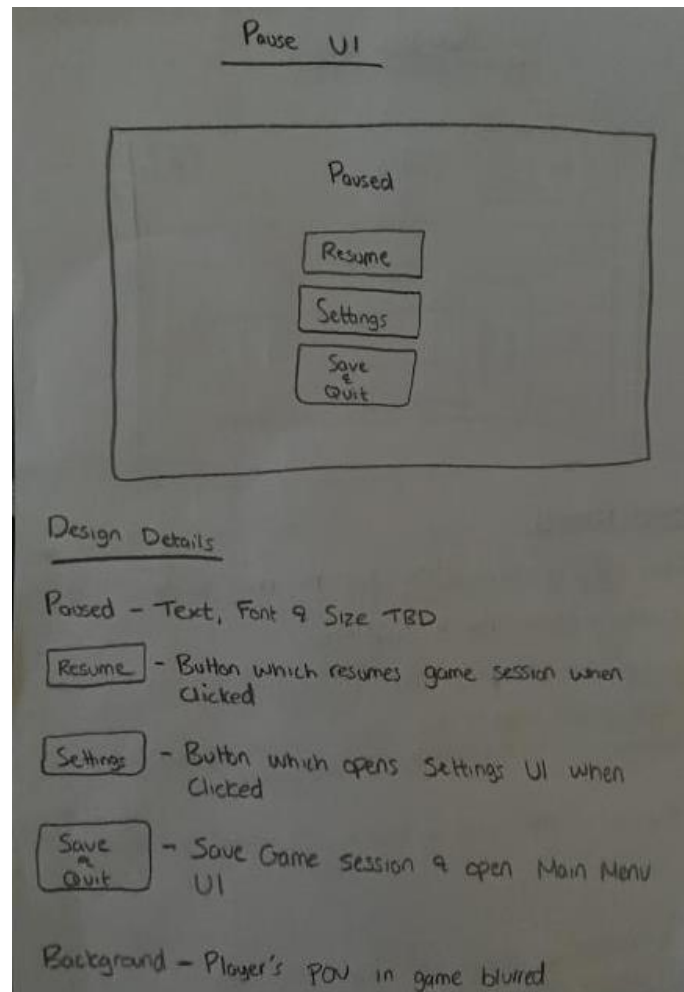


Figure 19. The pause UI

The pause UI is present when the player pauses the game while in a game session. The game physics will be paused while this UI is open. A resume button will allow the player to continue. They can open the game settings from here to make any changes they desire. The save and quit button will save the game data and return to the main menu. The background will be the player's current point of view in the world with the blur effect.

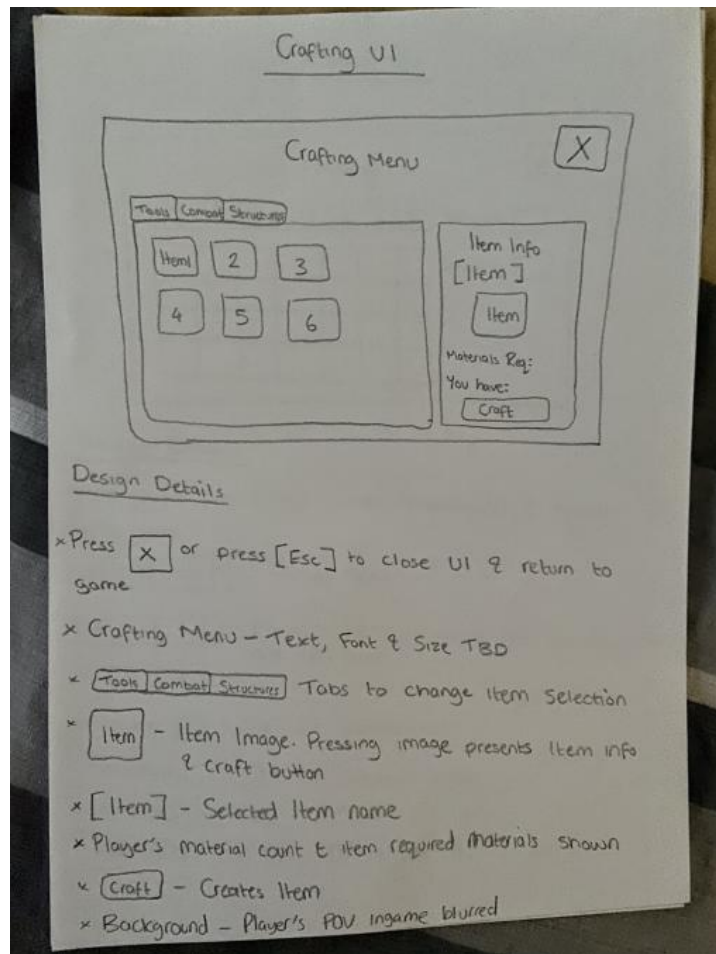


Figure 20. The crafting UI

The crafting UI allows the player to use their resources to craft items. The UI will feature tabs which categorise items. The player can select the items and see some item information. A button can then be pressed to craft item, deducting the resource cost and place the item into the player's inventory. The UI can be closed by the close button or pressing Esc.

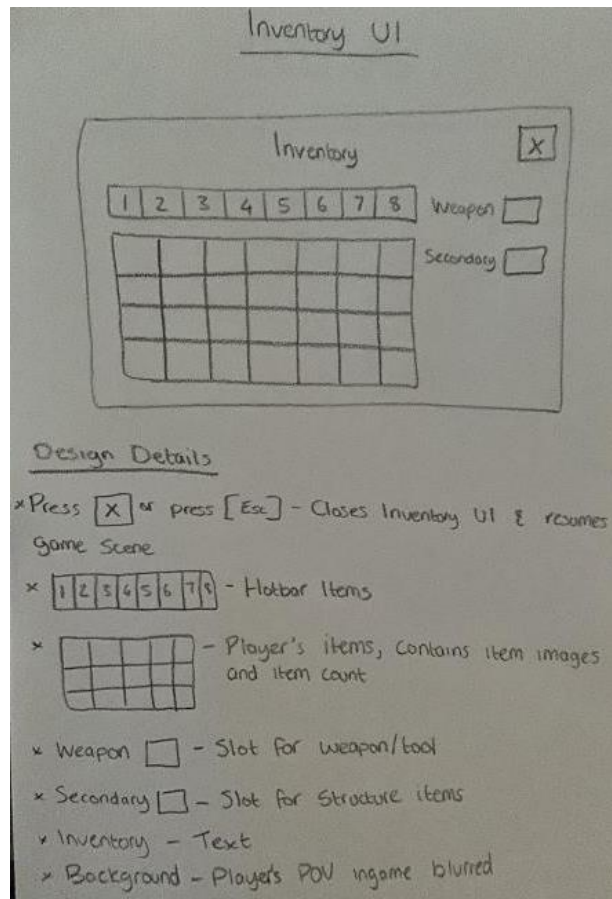


Figure 21. The inventory UI

The inventory UI shows the player their items. It features a grid-like UI show the item image and the amount they have. Access to the hot bar is provided which the player can add or remove items from there by a click-and-drag motion. A weapon and secondary item can be selected here as seen in the game scene UI. The UI can be closed by the close button or pressing Esc.

Indicative Test Plan

Testing

Testing is an integral part of the software development life cycle; it is the investigation about a feature or set of features to ensure the quality of product and ensure meets the expected requirements. Given the scale of this project, a large series of tests of both the engine and sandbox will be required to ensure all functional and non-functional requirements are met. The following set of paragraphs will discuss my testing strategy, the set of test cases with descriptions of what was testing and the data and the corresponding result of those tests.

Test Objectives

The project will organise the design of the software with the object-oriented programming model (OOP). There are many benefits for using the OOP model such as reusability, modularity, robustness, etc to name a few. This provides a clear path for testing as I can test these different categories and with the project made up of modules, I can test individual sections of the code without relying on others. I will be performing a variety of test methodologies such as unit testing and black box testing. Black box is a fast-testing approach where the internal structure and data is not known, and the tests determine whether a feature is working or not working from visual inspection. Unit testing is a more thorough and rugged testing methodology, testing specific components with the structure and data in the code being known and tested. The unit tests will confirm components working as they should in isolated instances and check for accuracy of data.

I will also be performing various performance profiling reports to check the performance of the prototype to check on CPU usage and memory usage. This ensures the quality of gameplay is as best it can be and is a good indication of the quality of code and data consumption by evaluating the overall usage but also profiling which functions are taking the most amount of CPU cycles.

Test Strategy

For the overall development of the project, I have decided to adopt the SCRUM agile methodology of software development. SCRUM allows for short sprints of development before performing inspections to check for any issues. I will be adopting the Agile software testing methodology as it allows the testing to occur periodically during development, testing small compositional modules rather than the whole system at the end of the development. This approach identifies bugs earlier on and creates a system that is strong and robust from the ground up. I will use the exploratory testing method which is more adaptable to changes and requires the tester (myself) to identify the functionality of a particular section of code and design and execute tests according to my understanding of what should be happening.

Unit Tests

For my prototype implementation, most of the work will be creating the engine which will support the game. I will be conducting various unit tests on the engine to ensure the correct

data is being created and manipulated. Screenshots of the results of the unit test will be provided after the test cases.

Unit Test Cases

Case	Summary	Expected Result	Passed?
1	Create a Window	Window is created with the properties given	
2	Close Window	Window is closed	
3	Get the Window properties	The properties of the window are retrieved	
4	Get Native Window	Gets the GLFW pointer to the window	
5	Get Window Position	Gets the position of the window	
6	Check graphics context is set	Checks the graphics context is valid	
7	Check window user data is set	Check the user data set in the window is a pointer to the event handler	
8	Check window properties after changing data	Updating the window size should update the window properties	
9	Check if window is minimised after minimising it	The window's minimised variable should be true after minimising the window	
10	Check the window on close event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
11	Check the window on resize event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
12	Check the window on focus event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
13	Check the window on lost focus event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
14	Check the window on moved event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
15	Check the window on key press event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
16	Check the window on key released event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
17	Check the window on key typed event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	

18	Check the window on mouse press event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
19	Check the window on mouse release event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
20	Check the window on mouse scroll event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
21	Check the window on movement event callback can be set	The default callback function returns false, once a new callback is set, the call back function should return true	
22	Check each event's event type, static type, and category flags	All event's match their event type, which should be the same as the static type and the category flag is set to the type of input (Key, Mouse, etc)	
23	Check the window close event can be handled	The window close event can return whether it has been handled and we can change this value with a function	
24	Check the window resize event stores the resize dimensions and can be handled	The window resize event can be probed for the new window size information	
25	Check the window focus event can be handled	The window focus event can return whether it has been handled and we can change this value with a function	
26	Check the window lost focus event can be handled	The window lost focus event can return whether it has been handled and we can change this value with a function	
27	Check the window move event stores the window's new position and can be handled	The window resize event can be probed for the new window position information	
28	Check the key press event for the key code of the key being pressed	The key press event can be probed to retrieve the keycode of the key being pressed	
29	Check the key release event for the key code of the key being released	The key release event can be probed to retrieve the keycode of the key being released	
30	Check the key typed event for the key code of the key being typed	The key typed event can be probed to retrieve the keycode of the key being typed	
31	Check the mouse press event for the mouse button being pressed	The mouse pressed event can be probed to retrieve the mouse button being pressed	

32	Check the mouse released event for the mouse button being released	The mouse released event can be probed to retrieve the mouse button being released	
33	Check the mouse move event stores the position of the mouse	The mouse moved event can be probed to retrieve the mouse position after it has been moved	
34	Check the mouse scrolled event stores the offset of the scroll button from its original position	The mouse scrolled event can be probed to retrieve the mouse scroll's current offset	
35	Can create a vertex buffer which has the correct layout and able to store data	A vertex buffer can be created with the layout of the data set and able to store an array of data in the GPU	
36	Can edit the data in a vertex buffer	The data in a vertex buffer can be edited by providing it the new data	
37	Can create and define layout of a vertex buffer	The layout is defined as an initialiser list and the layout can be probed to confirm its order, stride, and offsets	
38	Create an index buffer and store data	An index buffer can be created and the data inside the buffer can be set	
39	Can edit the data in an index buffer	The data in an index buffer can be edited by providing it the new data	
40	Can create a vertex array and add vertex buffers to it and set the index buffer	A vertex array can be created, and the vertex buffers can be bound to the array and so can an index buffer	
41	Can create a uniform buffer	A uniform buffer can be created, and a shader program can be attached. The buffer's layout can be probed.	
42	Can edit the data in a uniform buffer	The data in a uniform buffer can be edited by providing it the new data	
43	Can create an indirect buffer	An indirect buffer can be created, and a draw command can be added to the queue	
44	Can edit the data in an indirect buffer	The data in an indirect buffer can be edited by providing it the new data	
45	A 3D model can be loaded with a third-party library	An obj file can be loaded with assimp into custom data structures and upload the vertex data to the GPU	
46	A 2D model can be loaded (Batched)	A single 2D model can be loaded with all the data batched which has a limit of up 16 Quads (64 vertices)	
47	Check the initial data in the internal 2D renderer data is correct	The batch size for the 2D renderer should be 16, the draw count set to 0 and the vertices size should be 16x4 (64).	

48	The scene wide uniforms for 2D rendering should be set	Calling begin in the 2D renderer by providing a list of uniform buffers should set the scene wide uniforms in the internal data	
49	Submitting a 2D quad	Submitting a 2D quad should fill the first 4 vertices in the internal data's vertices with the quad data provided	
50	Check the initial data in the internal 3D renderer data is correct	The batch capacity should be 50 items, the vertex capacity is 50,000 and the index capacity is 25,000	
51	The vertex array in the internal 3D renderer data should be created	The vertex array will be created and set a valid identifier.	
52	The scene wide uniforms for 3D rendering should be set	Calling begin in the 3D renderer by providing a list of uniform buffers should set the scene wide uniforms in the internal data	
53	Submitting a 3D model	Submitting a 3D model should increase the batch queue by 1 and store the submission	
54	Creating a shader program and store the uniforms	A shader program should be created and set a valid identifier and the list of uniforms should be stored in the program	
55	Destroying a shader should wipe its data	The identifier for the program should be deleted and the resource cleaned up on OpenGL's side	
56	Checking the shader data type's information	Able to get the component count of data types, the size in memory and the alignment in memory	
57	Create a texture by providing the pixel data	A texture should be created and given an identifier, the pixel data stored in the GPU and stores various information like dimensions and channels	
58	Able to edit the pixel data of a texture	The data in GPU memory should change from the old data to the data provided by providing height and width of new data	
59	Able to create a subtexture	A subtexture can be created by providing the base texture its based on and UV start and end coordinates	
60	Able to transform model texture coordinates based on subtexture UV coordinates	The texture coordinates of a model can be transformed based on the UVs of a subtexture which is part of a whole texture	

61	Setting a texture unit for a texture	A texture unit manager will provide a texture with the next available texture unit slot	
62	Clear all units in texture unit manager	The buffer in the texture unit manager should all have all its data set to null	
63	Check when texture unit manager is full	The texture unit manager has a limited and a flag is set to true when this occurs.	
64	Loading a scene	A scene and all its associated data are loaded and set as the active scene in the SceneManager	
65	Destroying a scene	A scene and all its associated data are destroyed and removed from the SceneManager	
66	Adding a component to an entity	Adding a Mesh render component to an Entity which can be retrieved with a getter function	
67	Removing a component from an entity	Removing a Mesh render component to an Entity which removes component from component list	
68	Check if entity contains type of component	A boolean function returns whether the type of component exists within component list in the entity	
69	Get Entity transform	Creating a transform with data and adding it to an entity. Able to retrieve this transform and all the data	
70	Add entity to scene	Able to add an entity that has been created to a scene which adds the entity to the scene's entity list	
71	Check if entity name is taken	Checks if the entity list by name to see if name exists, name is used as an identifier so much be unique	
72	Remove entity from scene	Remove an entity from the scene entity list so the list decreases by one and deletes all the entity's data	
73	Set the main camera	Setting the scene's main camera which is used by the renderer for the render perspective	
74	Get and check camera properties	Create a camera with properties and check the properties match the data that was provided	
75	Get elapsed time	Reset a currently running timer and get the currently elapsed time	

76	Reset a timer	Resets a currently run timer and resets the elapsed time to 0	
77	Check a valid console logger can be created	Check the console logger pointer is valid and not null when the logger system is created	
78	Check a valid file logger can be created	Check the file logger pointer is valid when the file logging option is enabled	
79	Check a valid randomer system can be created	The randomiser system pointer should be a valid pointer when the system is started	
80	Generate random uniform values	Generate random values using uniform distribution for integers and floats within a range	
81	Generate random normal values	Generate random values using normal distribution for integers and floats providing a mean and standard deviation	
82	Adding a system to the system manager	A system can be added to the system manager's system list and automatically started	
83	Removing a system from the system manager	A system can be removed from the system manager's system list and automatically stopped	
84	Removing all systems from the system manager	All systems are be removed from the system manager's system list and automatically stopped	
85	Check if a system type is active	A boolean function which requires a system type which returns whether that type of system is currently active	
86	Get a list of all active systems	A list of all active systems is returned	
87	Time a function	Timer system is used which starts a timer, calls a function, then ends the timer and stores the elapsed time in a list	
88	Check if a valid timer system is created	The timer system pointer is valid, and the static enable boolean is set to true	
89	A timer can be created with a name through the timer system and can record times	A timer object is created with the name provided and can store a time in a list which can be retrieved	
90	Get a previously recorded time	Retrieve the time that was recorded by providing a valid timer name that was created	

91	Remove a timer	Deletes the timer with name provided from the list of timers in the system	
92	Get all times	Gets a list of all the times that have been recorded from the timer system	
93	Check if a timer exists	Check if a timer with name exists in the timer system	
94	Normalize some vector information into shorts	Transforms a vector of floats into array of shorts for small memory storage	
95	Packing vector data into 32-bit integer	Transform a vector of floats into a 32-bit integer	
96	Load resources into resource manager	All resources (Uniform buffers, shader programs, textures, sub textures, materials, and models) are loaded and stored in lists in the resource manager	

The figures below show off the various screenshots as proof of the success of the unit test cases.

```

----- Running 96 tests from 16 test suites.
----- Global test environment set-up.
----- 9 tests from WindowsTests
RUN WindowsTests.CreateWindow (613 ms)
OK WindowsTests.CloseWindow (142 ms)
RUN WindowsTests.WindowProperties (135 ms)
OK WindowsTests.GetNativeWindow (124 ms)
RUN WindowsTests.GetWindowPos (114 ms)
OK WindowsTests.GraphicsContext (53 ms)
RUN WindowsTests.WindowUserData (127 ms)
OK WindowsTests.WindowUpdateSize (128 ms)
RUN WindowsTests.Minimised (126 ms)
----- 9 tests from WindowsTests (1569 ms total)

----- 12 tests from EventHandler
RUN EventHandler.onClose (0 ms)
OK EventHandler.onResize (0 ms)
RUN EventHandler.onFocus (0 ms)
OK EventHandler.onLostFocus (0 ms)
RUN EventHandler.onWindowMoved (0 ms)
OK EventHandler.onKeyPressed (0 ms)
RUN EventHandler.onKeyReleased (0 ms)
OK EventHandler.onKeyTyped (0 ms)
RUN EventHandler.onMousePress (0 ms)
OK EventHandler.onMouseRelease (0 ms)
RUN EventHandler.onMouseMove (0 ms)
OK EventHandler.onMouseScroll (0 ms)
----- 12 tests from EventHandler (11 ms total)

```

Figure 22. Unit test results

```

----- 13 tests from Events
RUN Events.GetEventTypes (0 ms)
OK Events.CloseEvent (0 ms)
RUN Events.ResizeConstructor (0 ms)
OK Events.WindowsFocus (0 ms)
RUN Events.WindowsLostFocus (0 ms)
OK Events.WindowMoveConstructor (0 ms)
RUN Events.KeyPress (0 ms)
OK Events.KeyRelease (0 ms)
RUN Events.KeyTyped (0 ms)
OK Events.MouseMovedEvent (0 ms)
RUN Events.MousePressedEvent (0 ms)
OK Events.MouseReleasedEvent (0 ms)
RUN Events.MouseScrolledEvent (0 ms)
OK Events.MouseScrolledEvent (0 ms)
----- 13 tests from Events (13 ms total)

----- 10 tests from BufferTests
RUN BufferTests.CreateVertexBuffer (128 ms)
OK BufferTests.EditVertexBuffer (129 ms)
RUN BufferTests.VBufferLayout (129 ms)
OK BufferTests.CreateIndexBuffer (132 ms)
RUN BufferTests.EditIndexBuffer (125 ms)
OK BufferTests.CreateVertexArray (128 ms)
RUN BufferTests.CreateUniformBuffer (137 ms)
OK BufferTests.EditUniformBuffer (135 ms)
RUN BufferTests.CreateIndirectBuffer (129 ms)
OK BufferTests.EditIndirectBuffer (133 ms)
----- 10 tests from BufferTests (1318 ms total)

----- 2 tests from ModelTests
RUN ModelTests.CreateModel (135 ms)
OK ModelTests.Create2DModel (121 ms)
----- 2 tests from ModelTests (258 ms total)

```

Figure 23. Unit test results


```

-----] 3 tests from Renderer2DTests
RUN      Renderer2DTests.InternalData
      OK  Renderer2DTests.InternalData (878 ms)
RUN      Renderer2DTests.SettingSceneWideUniforms
      OK  Renderer2DTests.SettingSceneWideUniforms (751 ms)
RUN      Renderer2DTests.SubmitFirstQuad
      OK  Renderer2DTests.SubmitFirstQuad (749 ms)
-----] 3 tests from Renderer2DTests (2382 ms total)

-----] 4 tests from Renderer3DTests
RUN      Renderer3DTests.InternalData
      OK  Renderer3DTests.InternalData (754 ms)
RUN      Renderer3DTests.VAO
      OK  Renderer3DTests.VAO (756 ms)
RUN      Renderer3DTests.SettingSceneWideUniforms
      OK  Renderer3DTests.SettingSceneWideUniforms (752 ms)
RUN      Renderer3DTests.Submit3DModel
      OK  Renderer3DTests.Submit3DModel (757 ms)
-----] 4 tests from Renderer3DTests (3023 ms total)

-----] 3 tests from ShaderTests
RUN      ShaderTests.CreateShader
      OK  ShaderTests.CreateShader (122 ms)
RUN      ShaderTests.DestroyShader
      OK  ShaderTests.DestroyShader (39 ms)
RUN      ShaderTests.ShaderDataTypes
      OK  ShaderTests.ShaderDataTypes (0 ms)
-----] 3 tests from ShaderTests (166 ms total)

-----] 7 tests from TextureTests
RUN      TextureTests.CreateTexture
      OK  TextureTests.CreateTexture (59 ms)
RUN      TextureTests.EditTexture
      OK  TextureTests.EditTexture (124 ms)
RUN      TextureTests.CreateSubTexture
      OK  TextureTests.CreateSubTexture (129 ms)
RUN      TextureTests.SubTextureTransformUVs
      OK  TextureTests.SubTextureTransformUVs (131 ms)
RUN      TextureTests.SetTextureUnit
      OK  TextureTests.SetTextureUnit (132 ms)
RUN      TextureTests.ClearTextureUnitManager
      OK  TextureTests.ClearTextureUnitManager (142 ms)
RUN      TextureTests.FullUnitManager
      OK  TextureTests.FullUnitManager (129 ms)
-----] 7 tests from TextureTests (855 ms total)

```

Figure 24. Unit test results

```

-----] 11 tests from SceneTests
RUN      SceneTests.LoadScene
      OK  SceneTests.LoadScene (753 ms)
RUN      SceneTests.DestroyScene
      OK  SceneTests.DestroyScene (758 ms)
RUN      SceneTests.AddEntityComponent
      OK  SceneTests.AddEntityComponent (130 ms)
RUN      SceneTests.RemoveEntityComponent
      OK  SceneTests.RemoveEntityComponent (120 ms)
RUN      SceneTests.ContainsEntityComponent
      OK  SceneTests.ContainsEntityComponent (125 ms)
RUN      SceneTests.GetTransform
      OK  SceneTests.GetTransform (136 ms)
RUN      SceneTests.AddEntityToScene
      OK  SceneTests.AddEntityToScene (756 ms)
RUN      SceneTests.EntityNameTaken
      OK  SceneTests.EntityNameTaken (758 ms)
RUN      SceneTests.RemoveEntityFromScene
      OK  SceneTests.RemoveEntityFromScene (766 ms)
RUN      SceneTests.SettingMainCamera
      OK  SceneTests.SettingMainCamera (757 ms)
RUN      SceneTests.CameraProperties
      OK  SceneTests.CameraProperties (764 ms)
-----] 11 tests from SceneTests (5836 ms total)

-----] 2 tests from Timer
RUN      Timer.ElapsedTime
      OK  Timer.ElapsedTime (1 ms)
RUN      Timer.ResetTime
      OK  Timer.ResetTime (1 ms)
-----] 2 tests from Timer (2 ms total)

-----] 2 tests from LogSystem
RUN      LogSystem.ConsoleLogger
      OK  LogSystem.ConsoleLogger (0 ms)
RUN      LogSystem.FileLogger
      OK  LogSystem.FileLogger (1 ms)
-----] 2 tests from LogSystem (4 ms total)

-----] 3 tests from RandomiserSystem
RUN      RandomiserSystem.System
      OK  RandomiserSystem.System (0 ms)
RUN      RandomiserSystem.GenerateUniforms
      OK  RandomiserSystem.GenerateUniforms (0 ms)
RUN      RandomiserSystem.GenerateNormals
      OK  RandomiserSystem.GenerateNormals (0 ms)
-----] 3 tests from RandomiserSystem (3 ms total)

```

Figure 25. Unit test results

```

-----] 6 tests from SystemManagerTests
RUN      SystemManagerTests.AddSystems
      OK  SystemManagerTests.AddSystems (0 ms)
RUN      SystemManagerTests.RemoveSystems
      OK  SystemManagerTests.RemoveSystems (0 ms)
RUN      SystemManagerTests.RemoveAllSystems
      OK  SystemManagerTests.RemoveAllSystems (1 ms)
RUN      SystemManagerTests.IsSystemActive
      OK  SystemManagerTests.IsSystemActive (1 ms)
RUN      SystemManagerTests.GetAllSystems
[SystemManager::getRandomiser] Randomiser system is not active.
[SystemManager::getTimerSystem] Timer system is not active.
      OK  SystemManagerTests.GetAllSystems (1 ms)
RUN      SystemManagerTests.TimeFunction
      OK  SystemManagerTests.TimeFunction (1 ms)
-----] 6 tests from SystemManagerTests (8 ms total)

-----] 6 tests from TimerSystemTests
RUN      TimerSystemTests.System
      OK  TimerSystemTests.System (0 ms)
RUN      TimerSystemTests.RecordingTimes
      OK  TimerSystemTests.RecordingTimes (0 ms)
RUN      TimerSystemTests.GetTime
      OK  TimerSystemTests.GetTime (0 ms)
RUN      TimerSystemTests.RemoveTimer
      OK  TimerSystemTests.RemoveTimer (0 ms)
RUN      TimerSystemTests.GetAllTimes
      OK  TimerSystemTests.GetAllTimes (0 ms)
RUN      TimerSystemTests.TimerExists
      OK  TimerSystemTests.TimerExists (0 ms)
-----] 6 tests from TimerSystemTests (6 ms total)

-----] 3 tests from UtilityTests
RUN      UtilityTests.Normalize
      OK  UtilityTests.Normalize (0 ms)
RUN      UtilityTests.Packing
      OK  UtilityTests.Packing (0 ms)
RUN      UtilityTests.ResourceManager
      OK  UtilityTests.ResourceManager (742 ms)
-----] 3 tests from UtilityTests (746 ms total)

-----] Global test environment tear-down
-----] 96 tests from 16 test suites ran. (16156 ms total)
PASSED  ] 96 tests.

Press enter to continue

```

Figure 26. Unit test results

Black box testing

With the focus of the prototype being working on the engine. To show off some visual aspect is a little tricky. There is not much functionality to look at, however I will create a sample scene with a functioning camera. Black box testing will allow me to check the success of this implementation.

Case	Summary	Expected Results	Passed
1	Test camera controls move the camera	Able to use the W,A,S,D keys to traverse the scene based on the front direction vector of the camera	
2	Test the blinn-phong lighting model with attenuation	A blinn-phong model was added and a light added to the world. The white light has an origin and should dissipate over distance using a non-linear equation	
3	A 3D model should be rendered to the screen	A 3D model is visible in the 3D world	
4	A 2D model should be rendered to the screen	A 2D model should be visible onscreen appearing above 3D environment	
5	Text should be rendered to the screen	Text should be rendered to the screen with the 2D renderer	

Implementation

Software Implementation

The current implementation of the software largely focuses on the engine required to produce the game set out in my project contract and system design. A rendering system is required to render 3D and 2D geometry with the various systems in place to manage assets, shader programs, and the data associated with rendering and required by OpenGL.

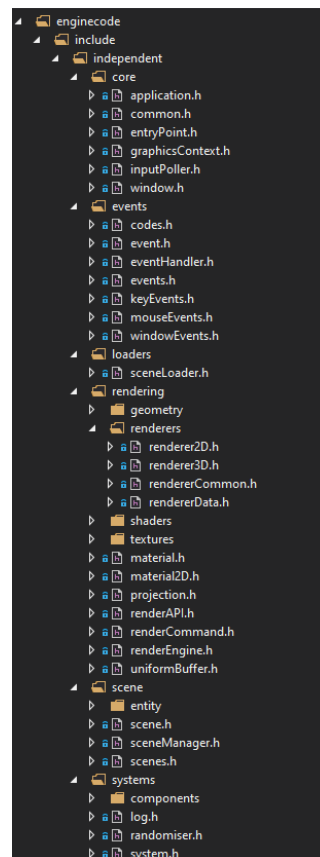


Figure 27. The workspace of the engine

As you can see from the figure above, the workspace for the engine is growing with the requirements for the engine being fulfilled and a lot of work is required to get the engine completed.

A sample scene has been provided which shows off the rendering of 3D geometry, 2D geometry and text rendering. OBJ files containing various 3D geometry can be loaded through my custom assimp loader and are then rendered to the sample scene. Blinn-phong lighting model was added to the shader program used which uses ambient, diffuse, and specular components followed by applying an attenuation equation which is demonstrated in the figure below.

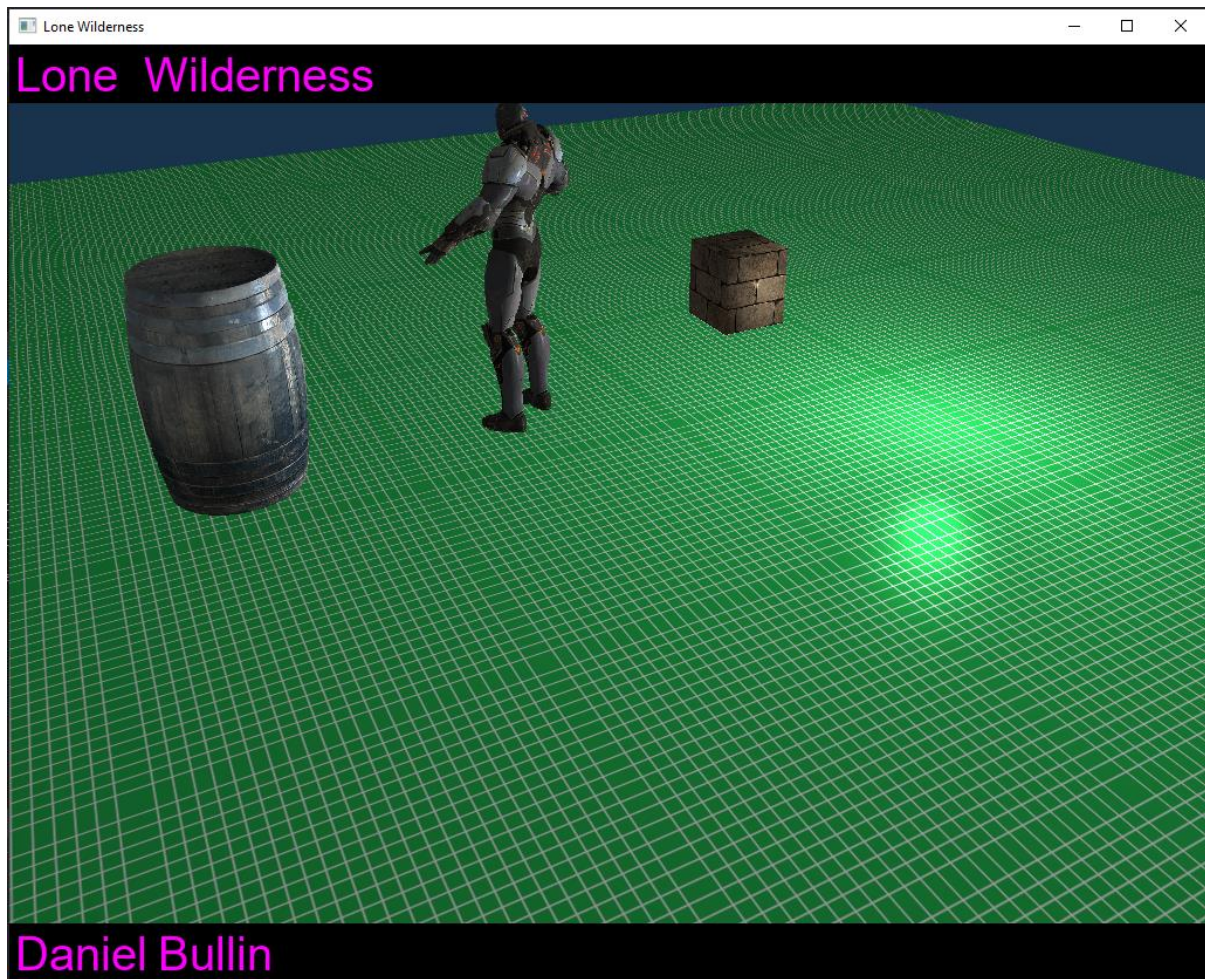


Figure 28. A sample scene using the current engine implementation

Scene data is stored in files in JSON format and are loaded by the scene loader class I have implemented. This offers a nice, neat way to implement scene data and keep it separate from the source code.

```
{
  "entities":
  [
    {
      "name": "Terrain",
      "components":
      [
        {
          "name": "Transform1",
          "type": "Transform",
          "transform": [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0 ]
        },
        {
          "name": "MeshRender1",
          "type": "MeshRender",
          "modelName": "terrain",
          "materialName": "gridMaterial"
        }
      ]
    },
    {
      "name": "cyborg",
      "components":
      [
        {
          "name": "Transform1",
          "type": "Transform",
          "transform": [ 0.0, 0.0, -6.0, 0.0, 0.0, 0.0, 2.0, 2.0, 2.0 ]
        },
        {
          "name": "MeshRender1",
          "type": "MeshRender",
          "modelName": "cyborg",
          "materialName": "cyborgMaterial"
        }
      ]
    }
  ]
}
```

Figure 29. An excerpt of the entity scene data file

Resources are loaded in a similar way by being stored in JSON formatted files.

```
"2DTextures": [
  { "textureName": "default", "filePath": "assets/textures/default.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false},
  { "textureName": "border", "filePath": "assets/textures/border.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false},
  { "textureName": "grid", "filePath": "assets/textures/grid.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": true},
  { "textureName": "text", "filePath": "assets/textures/text.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false},
  { "textureName": "cyborg_diffuse", "filePath": "assets/textures/cyborg/cyborg_diffuse.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false},
  { "textureName": "brickwall_diffuse", "filePath": "assets/textures/brickwall/brickwall_diffuse.jpg", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false},
  { "textureName": "letterAndNumbers_diffuse", "filePath": "assets/textures/letterAndNumberCube.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false},
  { "textureName": "barrel_diffuse", "filePath": "assets/textures/barrel/barrel.png", "wrapS": 10497, "wrapT": 10497, "wrapR": 10497,
    "minFilter": 9729, "maxFilter": 9729, "gammaCorrect": false, "flipUV": false}
]
```

Figure 30. An exert of the textures file of the textures to be loaded

A lot of functionality has been implemented, one of those features is batched rendering which offers a more efficient way of rendering. Submissions are added to the back of a batch queue to be rendered with a single draw call using indirect buffers.

```
///! submit()
///!
\param meshRender a MeshRender* - The render details of the entity
\param transformationMatrix a const glm::mat4& - The model matrix
*/
void Renderer3D::submit(MeshRender* meshRender, const glm::mat4& transformationMatrix)
{
    // Check queue size and add submission
    if (s_data->batchQueue.size() == s_data->BatchCapacity)
        flushBatch();
    else
        s_data->batchQueue.push_back({ meshRender, transformationMatrix });
}
```

Figure 31. Adding 3D geometry submission to batch queue

```

//! flushBatchCommands()
//!
\param shader a const std::shared_ptr<ShaderProgram>& - The shader program
\param instanceCount a uint32_t - The number of instances to draw
*/
void Renderer3D::flushBatchCommands(const std::shared_ptr<ShaderProgram>& shader, uint32_t instanceCount)
{
    shader->start();

    // Bind the uniform buffer to the current shader
    for (auto& dataPair : s_data->sceneWideUniforms)
    {
        const char* nameOfUniformBlock = dataPair.first;
        dataPair.second->attachShaderBlock(shader, nameOfUniformBlock);
    }

    // Upload texture units
    shader->sendIntArray("u_diffuseMap", s_data->textureUnit.data(), 16);

    // Edit all instance data in GPU memory
    auto VBOModels = s_data->VAO->getVertexBuffers().at(1);
    auto VBOTexUnit = s_data->VAO->getVertexBuffers().at(2);
    auto VBOTint = s_data->VAO->getVertexBuffers().at(3);
    VBOModels->edit(s_data->modelInstanceData.data(), sizeof(glm::mat4) * instanceCount, 0);
    VBOTexUnit->edit(s_data->texUnitInstanceData.data(), sizeof(uint32_t) * instanceCount, 0);
    VBOTint->edit(s_data->tintInstanceData.data(), sizeof(uint32_t) * instanceCount, 0);

    // Bind VAO
    s_data->VAO->bind();
    // Edit Indirect buffer
    s_data->commands->edit(s_data->batchCommandsQueue.data(), s_data->batchCommandsQueue.size(), 0);

    // Draw
    s_data->VAO->drawMultiIndirect(s_data->batchCommandsQueue.size());
}

```

Figure 32. Flushing the batch queue to draw geometry